

Optimizavimas be apribojimų

Lukaš Patrik Magalinski

Spalis 2024

1) Įvadas

Laboratorinio darbo tikslas - pasinaudoti gradientinio nusileidimo, greičiausiojo nusileidimo, deformuojamo simplekso algoritmomis, kad išspręsti optimizavimo uždavinį ir palyginti rezultatus skirtinguose pradiniuose taškuose: sprendinius, funkcijos minimumo iverčius, atliktų žingsnių kiekius, funkcijų skaičiavimų skaičius.

2) Optimizavimo uždavinys

Turime stačiakampio gretasienio formos dėžę su vienetine paviršiaus plotu. Optimizavimo uždavinys yra tame kad rasti maksimalųjį turį šiai dėžei.

Pažyme kime ilgį a , plotį b ir aukštį c ir apibreškime dėžės paviršiaus plotą:

$$x = S_{\text{priekinė+galinė}} = 2ac$$

$$y = S_{\text{šoninės}} = 2cb$$

$$z = S_{\text{viršutinė+apatinė}} = 2ab$$

$$S_{\text{paviršiaus}} = x + y + z = 1$$

Išreiškime z per dėžės paviršiaus plotą: $z = 1 - x - y$

Dėžės tūris pakeltas kvadratu:

$$V^2 = a^2 b^2 c^2 = \left(\frac{1}{2}x\right) ab^2 c = \left(\frac{1}{2}x\right) \left(\frac{1}{2}y\right) ab = \left(\frac{1}{2}x\right) \left(\frac{1}{2}y\right) \left(\frac{1}{2}z\right) = \frac{xyz}{8}$$

$$\frac{xyz}{8} = \frac{xy(1-x-y)}{8} = \frac{xy - x^2y - xy^2}{8}$$

Apibreškime minimizavimo funkciją $V(x, y)$:

$$\min V(x, y) = -\frac{xy - x^2y - xy^2}{8} = \frac{x^2y + xy^2 - xy}{8} = 0.125 * (x^2y + xy^2 - xy)$$

Ieškosime $\nabla V(x, y)$: $\nabla V(x, y) = \left(\frac{\partial V}{\partial x}, \frac{\partial V}{\partial y}\right)$

$$V(x, y) = \frac{x^2y + xy^2 - xy}{8}$$

$$\frac{\partial V}{\partial x} = \frac{1}{8} * (2xy + y^2 - y) = \frac{y(2x + y - 1)}{8}$$

$$\frac{\partial V}{\partial y} = \frac{1}{8} * (2xy + x^2 - x) = \frac{x(2y + x - 1)}{8}$$

Iš $\frac{\partial V}{\partial x}$ ir $\frac{\partial V}{\partial y}$ mes galim gauti $\nabla V(x, y)$:

$$\nabla V(x, y) = \left(\frac{y(2x + y - 1)}{8}, \frac{x(2y + x - 1)}{8} \right)$$

Optimizavimo algoritmus taikysime trijuose pradinuose taškuose:

1) $X_0 = (0, 0)$

2) $X_1 = (1, 1)$

3) $X_m = \left(\frac{a}{10}, \frac{b}{10} \right)$, kur a, b – paskutiniai du studento pažymėjimo numerio skaitmenys. Mano studento pažymėjimo numeris: [redacted]. Iš to, $a = 0, b = 4$. Iš to seka: $X_m = \left(\frac{0}{10}, \frac{4}{10} \right) = \left(0, \frac{4}{10} \right) = (0, 0.4)$

Apskaičiuokime $V(x, y)$ ir $\nabla V(x, y)$ reikšmes šiuose taškuose:

1) $X_0 = (0, 0)$:

$$V(0, 0) = 0$$

$$\nabla V(0, 0) = (0, 0)$$

2) $X_1 = (1, 1)$:

$$V(1, 1) = \frac{1}{8}$$

$$\nabla V(1, 1) = \left(\frac{1}{4}, \frac{1}{4} \right)$$

3) $X_m = (0, 0.4)$:

$$V(0, 0.4) = 0$$

$$\nabla V(0, 0.4) = \left(-\frac{3}{100}, 0 \right)$$

3) Gradiento nusileidimo algoritmas

Gradientinio nusileidimo algoritmas kiekvieną tašką apskaičiuoja pagal formulę:

$$X_{i+1} = X_i - \gamma \nabla V(X_i),$$

kur $X := (x, y)$, γ - žingsnio daugiklis, konstanta.

Algoritmas baigia savo darbą, kai gradiento norma yra mažiau už $\varepsilon = 0.0001$ arba per ilgai ieško minimumą. Maksimum iteracijų skaičius yra 10000.

Parašiau Python programą (žr. Priedas 7.1), kuri randa lokalių minimumo tašką naudodama gradientinio nusileidimo algoritmą (Python kode: `def gradient_descent`).

3.1) $X_1 = (1, 1)$

Gradientinis nusileidimas, pradedant nuo $X_1 = (1, 1)$, $\varepsilon = 0.0001$				
γ	$X_{\min}$	Iteracijų skaičius	Apskaičiuotų gradientų skaičius	Funkcijos $V(x, y)$ reikšmė:
0.03	(0.3339, 0.3339)	1590	1591	-0.004630
0.06	(0.3339, 0.3339)	793	794	-0.004630
0.12	(0.33389, 0.33389)	395	396	-0.004630
0.2666	(0.33388, 0.33388)	176	177	-0.004630
0.5331	(0.33388, 0.33388)	86	87	-0.004630
0.7777	(0.33385, 0.33385)	58	59	-0.004630
1.333	(0.33384, 0.33384)	32	33	-0.004630
1.5996	(0.33381, 0.33381)	25	27	-0.004630
1.8667	(0.33386, 0.33386)	21	22	-0.004630
2.4112	(0.3338, 0.3338)	14	15	-0.004630
2.666	(0.33325, 0.33325)	1	2	-0.004630
2.999	(0.33287, 0.33287)	13	14	-0.004630
3.239	(0.33292, 0.33292)	14	15	-0.004630
3.7233	(0.333, 0.333)	16	17	-0.004630
3.999	(0.00003, 0.00003)	1	2	-0.004630
4.01	diverguoja į $-\infty$	-	-	-
4.5121	diverguoja į $-\infty$	-	-	-

Po tyrimo su skirtingomis γ ir pastovu $\varepsilon = 0.0001$ yra pastebėjimų:

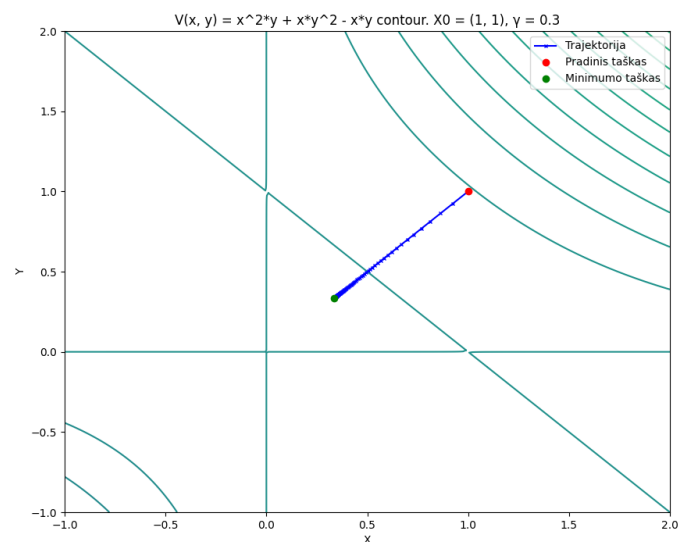
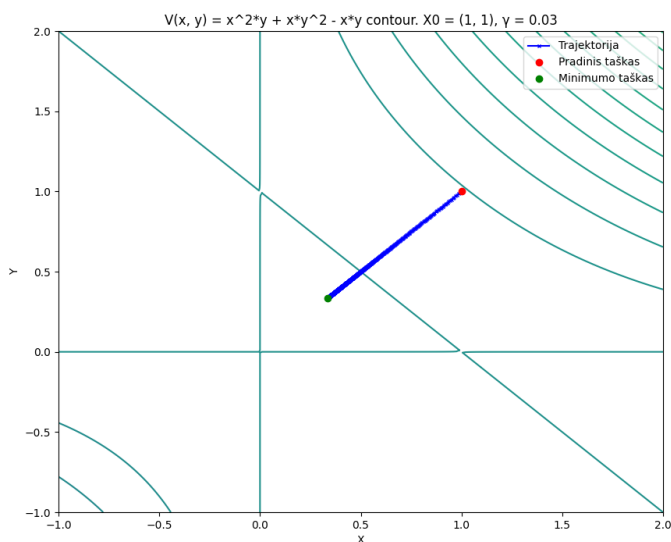
- Daugeliu atvejų minimumo taškas $x_{\min}$ konverguoja iki (0.(3),0.(3)).
- Kai $\gamma = 2.(6)$, algoritmas baigė savo darbą per vieną iteraciją. Matosi, kad artėjant γ iki 2.(6), mažėja ir iteracijų kiekis, ir apskaičiuotų gradientų skaičius (išskyrus atvejį su $\gamma = 3.999$).
- Kai $\gamma = 3.999$, algoritmas baigė savo darbą po pirmos iteracijos ir surado blogą minimumą tašką. Tai atsitiko dėl to, kad $\varepsilon = 0.0001$ yra per didelis.

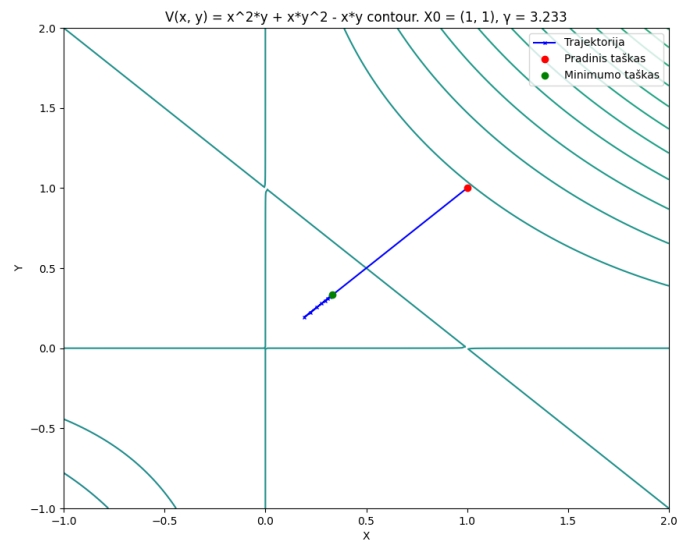
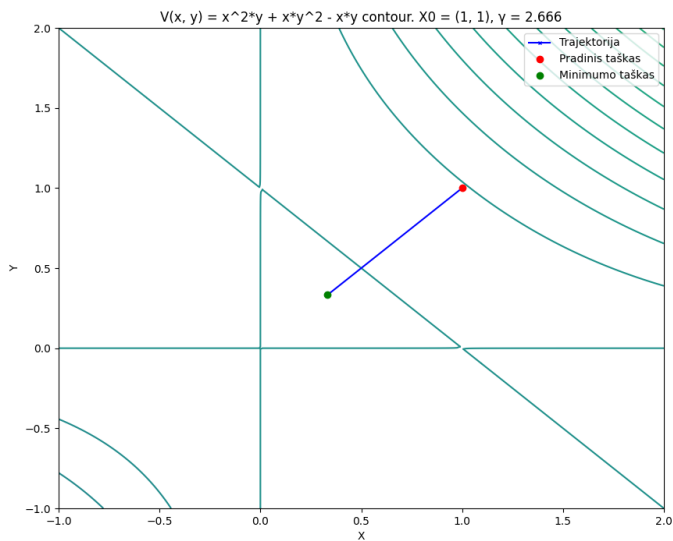
$$\nabla V(0.00003, 0.00003) = (-0.00000375, -0.00000375)$$

$$|\nabla V(0.00003, 0.00003)| = 0.0000053 < \varepsilon$$

Sumažinus ε iki 0.00000001, algoritmas pratęs savo darbą toliau, ir galų gale duos $x_{\min} = (0.33333, 0.33333)$ per 48 iteracijas ir 49 apskaičiuotų gradientų. Žingsniai po 3.999 yra per dideles ir dėl to $x_{\min}$ diverguoja į $-\infty$.

Žemiau pateikiau minimumo taško suradimo grafikus pradedant nuo X_1 taško su skirtingomis γ reikšmėmis:





3.2) $X_m = (0, 0.4)$

Gradientinis nusileidimas, pradedant nuo $X_1 = (1, 1)$, $\varepsilon = 0.0001$				
γ	$X_{\min}$	Iteracijų skaičius	Apskaičiuotų gradientų skaičius	Funkcijos $V(x, y)$ reikšmė:
0.03	(0.33164, 0.33504)	3629	3630	-0.004630
0.5	(0.33167, 0.33501)	216	217	-0.004630
1.1	(0.33169, 0.33499)	97	98	-0.004630
1.723	(0.3317, 0.33498)	61	62	-0.004630
2.53	(0.33176, 0.33492)	41	42	-0.004630
3.455	0.33171, 0.33497)	29	30	-0.004630
4.632	(0.33176, 0.33492)	21	22	-0.004630
5.123	(0.33187, 0.3348)	19	20	-0.004630
6.555	(0.33183, 0.33485)	14	15	-0.004630
7.226	(0.33214, 0.33453)	13	14	-0.004630
7.777	(0.33171, 0.33497)	11	12	-0.004630
9.4	(0.33218, 0.33449)	9	10	-0.004630
10.523	(0.33193, 0.3348)	7	8	-0.004630
11.990	(0.33278, 0.33312)	8	9	-0.004630
12.7832	(0.3328, 0.33282)	10	11	-0.004630
14.923	(0.33385, 0.33385)	29	30	-0.004630
15.4523	(0.33278, 0.33278)	48	49	-0.004630
15.99123	(0.33277, 0.33277)	1457	1458	-0.004630
16.124	(0.35268, 0.35268)	10000(riba pasiekta)	10001	-0.004581
16.424	(0.36666, 0.36666)	10000(riba pasiekta)	10001	-0.004482
17.678	(0.38839, 0.38839)	10000(riba pasiekta)	10001	-0.004209
17.923	(0.39056, 0.39056)	10000(riba pasiekta)	10001	-0.004173
18.2	(0.23375, 0.23375)	10000(riba pasiekta)	10001	-0.003637
18.80358	(0.39604, 0.39604)	10000(riba pasiekta)	10001	-0.004077
>19	diverguoja į $-\infty$	-	-	-

Po tyrimo su skirtingomis γ ir pastovu $\varepsilon = 0.0001$ yra pastebėjimų:

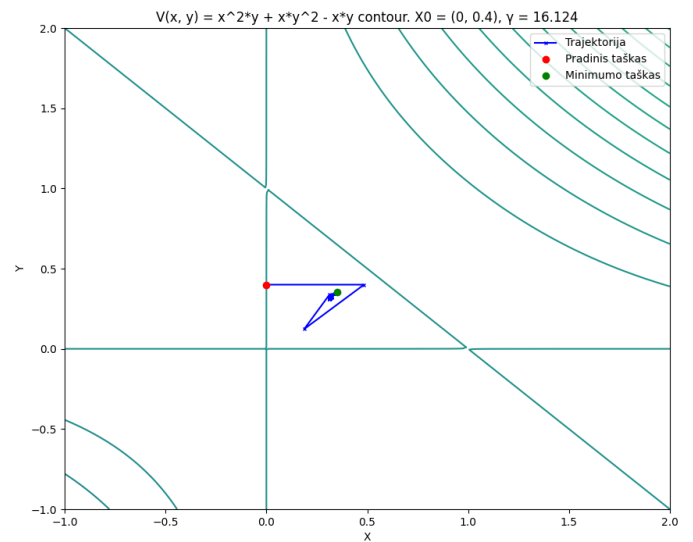
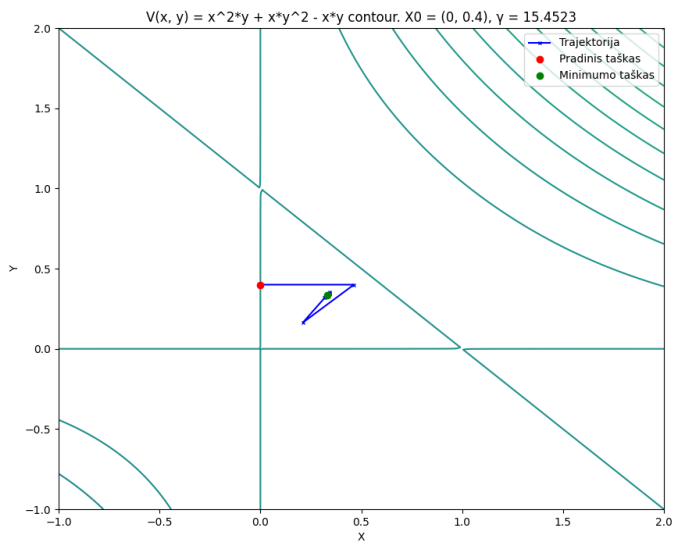
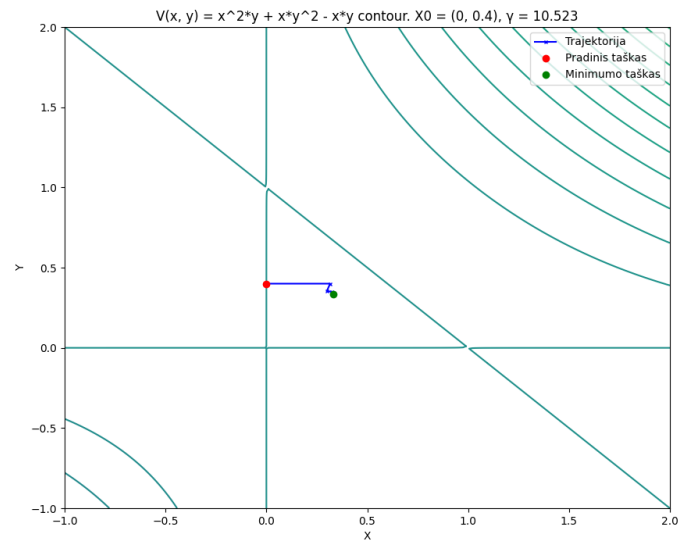
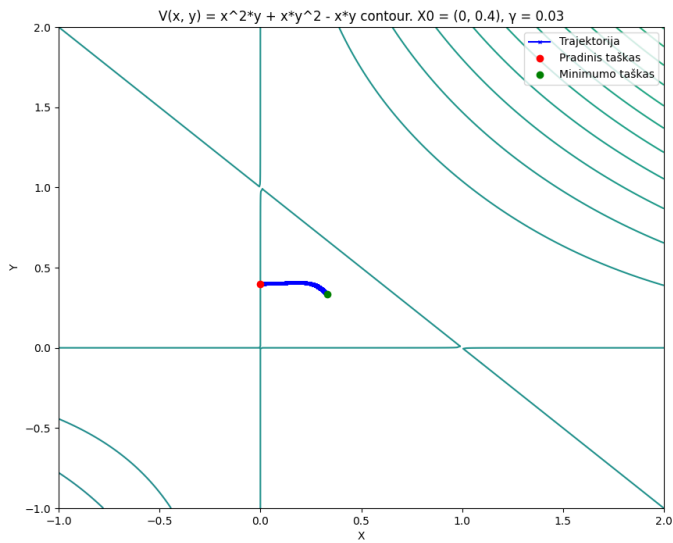
- Daugelių atveju, minimumo taškas $x_{\min}$ konverguoja iki (0.3),0.3))
- $\gamma = 10.523$ per 7 iteracijas rado minimumą tašką. Tačiau nėra tokios γ , kad algoritmas baigtų savo darbą per vieną iteraciją.
- Žingsniai po 19 yra per dideles ir dėl to $x_{\min}$ diverguoja į $-\infty$

Yra tokios γ reikšmės su kuriais algoritmas per ilgai ieško minimumą tašką. Tokiais atvejais prie iteracijos skaičiaus prirašyta "(riba pasiekta)". Iš tikrųjų, su tokiais γ reikšmėmis algoritmas niekada nepasieks minimumą, nes gradiento ilgis labai lėtai mažėja ir galu gale algoritmas užstrigs tarp dviejų taškų. Pavyzdžiui, paimsime $\gamma = 16.124$. Lentelėje žemiau nurodžiau kaip keičiasi x -ų ir gradientų ilgių reikšmės:

$X_m = (0, 0.4), \gamma = 16.124$			
Iteracija	$X_{\min}$	Gradiento ilgis	Funkcijos $V(x, y)$ reikšmė:
0	(0, 0.4)	0.030000	0.000000
1	(0.483720, 0.400000)	0.025136	-0.002812
2	(0.187490, 0.123391)	0.015351	-0.001993
3	(0.312242, 0.337172)	0.001699	-0.004614
4	(0.338300, 0.345614)	0.001575	0.004620
5	(0.322826, 0.325481)	0.001581	0.004619
6	(0.341763, 0.342536)	0.001600	-0.004620
7	(0.323770, 0.324052)	0.001619	-0.004619
8	(0.342324, 0.342406)	0.001640	-0.004619
9	(0.323654, 0.323683)	0.001659	-0.004618
10	(0.342579, 0.342587)	0.001681	-0.004619
...	...	...	...
480	(0.352682, 0.352682)	0.003619	-0.004581
481	(0.311421, 0.311421)	0.003619	-0.004572
482	(0.352682, 0.352682)	0.003619	-0.004581
483	(0.311421, 0.311421)	0.003619	-0.004572
484	(0.352682, 0.352682)	0.003619	-0.004581
485	(0.311421, 0.311421)	0.003619	0.004572
486	(0.352682, 0.352682)	0.003619	-0.004581
487	(0.311421, 0.311421)	0.003619	-0.004572
488	(0.352682, 0.352682)	0.003619	-0.004581
489	(0.311421, 0.311421)	0.003619	-0.004572
490	(0.352682, 0.352682)	0.003619	-0.004581
...	...	...	...
9990	(0.352682, 0.352682)	0.003619	-0.004581
9991	(0.311421, 0.311421)	0.003619	-0.004572
9992	(0.352682, 0.352682)	0.003619	-0.004581
9993	(0.311421, 0.311421)	0.003619	-0.004572
9994	(0.352682, 0.352682)	0.003619	-0.004581
9995	(0.311421, 0.311421)	0.003619	-0.004572
9996	(0.352682, 0.352682)	0.003619	-0.004581
9997	(0.311421, 0.311421)	0.003619	-0.004572
9998	(0.352682, 0.352682)	0.003619	-0.004581
9999	(0.311421, 0.311421)	0.003619	-0.004572
10000	(0.352682, 0.352682)	0.003619	-0.004581

Matosi, kad algoritmas šokinėja tarp taškų (0.352682, 0.352682) ir (0.311421, 0.311421). Gradiento ilgis nustojo mažėti nuo 0.003619, o tai yra daugiau už ε . Algoritmas pateko į begalinę ciklą ir negali baigti savo darbą. Judant per dideliais žingsniais, algoritmas gali peršokti per minimumą ir pradėti šokinėti tarp taškų, niekada nesikonverguodamas į minimumą, o mūsų žingsnis ($\gamma = 16.124$) yra per didelis, kitaip sakant algoritmas atnaujiną tašką, bet dėl per didelio žingsnio dydžio peršoka per minimumą, o kitoje iteracijoje grįžta atgal, sukuriant begalinį ciklą. Dar, jeigu pažiūrėti į gradiento ilgį (gradiento norma) tai jei gradiento norma nustoja mažėti, tai reiškia, kad algoritmas nebeturi galimybių priartėti arčiau minimumo su dabartiniu žingsnio dydžiu. Todėl galima pasakyti, kad geriau paimti mažesnę γ , pavyzdžiui 0.5, ir dar galima sumažinti $\varepsilon = 0.000001$ ir su šiais duomenimis $X_{\min} \Rightarrow (0.33332, 0.33335)$.

Žemiau pateikiau mimumo taško suradimo grafikus pradant nuo X_m taško su skirtingomis γ reikšmėmis:



3.3) $X_0 = (0, 0)$

$$\nabla V(0, 0) = (0, 0) \Rightarrow |\nabla V(0, 0)| = 0 < \varepsilon$$

Algoritmas nepradės dirbti su tašku $(0, 0)$, nes gradiento ilgis šiame taške lygus 0, o tai yra mažiau už ε

4) Greičiausio nusileidimo algoritmas

Greičiausiojo nusileidimo algoritmas kiekvieną tašką apskaičiuoja pagal formulę:

$$X_{i+1} = X_i - \gamma_i \nabla V(X_i),$$

kur $X := (x, y)$, γ_i - žingsnio daugiklis.

Žingsnio daugiklis yra perskaičiuojamas kiekvienoje iteracijoje, naudojant vienmačio minimizavimo algoritmą:

$$\gamma_i = \arg \min_{\gamma \geq 0} f(\gamma),$$

kur $f(\gamma) = V(X_i - \gamma \nabla f(X_i))$. Įstatęs γ_i į formulę, gauname:

$$X_{i+1} = X_i - \arg \min_{\gamma \geq 0} V(X_i - \gamma \nabla f(X_i)) \nabla V(X_i)$$

Mano atveju greičiausiojo nusileidimo algoritmas naudojo aukso pjūvio algoritmą. Algoritmas baigia savo darbą, kai gradiento norma yra mažiau už $\varepsilon = 0.0001$, galima naudoti ir mažesnius ε reikšmės, pvz.: $\varepsilon = 0.000001$, $\varepsilon = 0.00000001$.

Parašiau Python programą (žr. Priedas 7.1), kuri randa lokaliųjį minimumo tašką naudodama greičiausiojo nusileidimo algoritmą (Python kode: `def gradient_descent_optimal`).

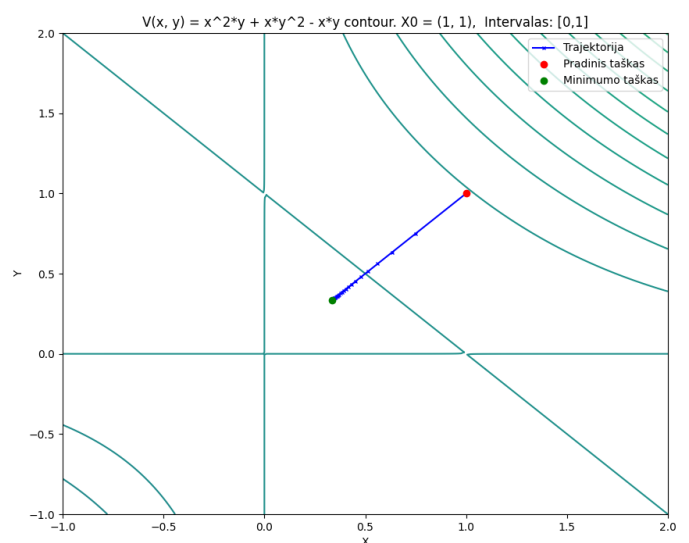
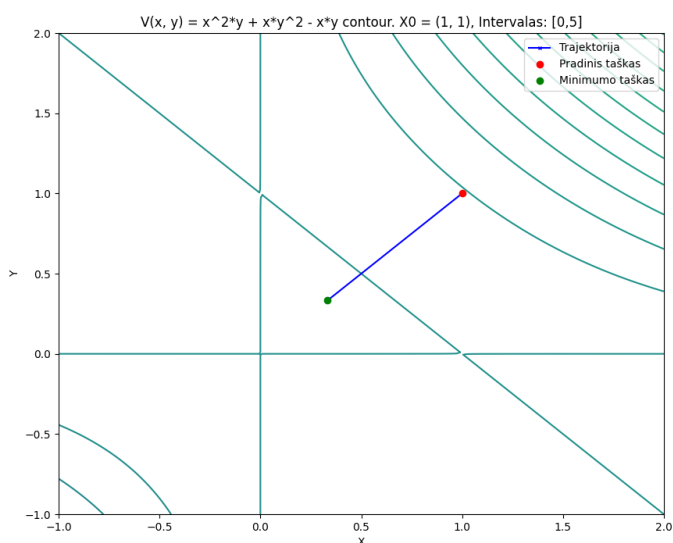
Algoritmas, naudodamas **aukso pjūvio algoritmą**, kiekvienoje iteracijoje nustato optimaliausią žingsnio dydį. Taigi, aukso pjūvio intervalai, kuriuos naudosiu, yra $[0,1]$, $[0, 5]$, $[5, 10]$, $[20, 30]$ ir kiti. Tai yra intervalai kuriose aukso pjūvio algoritmas ieško minimumo.

4.1) $X_1 = (1, 1)$

Greičiausio nusileidimo algoritmas, pradedant nuo $X_1 = (1, 1)$, $\varepsilon = 0.0001$						
l	r	γ	$X_{\min}$	Iteracijų skaičius	V(x, y) iškvietimų skaičius	Gradientų skaičius
0	1	0.999966	(0.33386, 0.33386)	44	1013	45
0	5	2.666660	(0.33333, 0.33333)	1	28	2
0	10	0.014563	diverguoja į $-\infty$	-	-	-
1	5	2.66665	0.33334, 0.33334)	1	26	2

Algoritmas baigė savo darbą per vieną iteraciją intervale $[0; 5]$, radus $\gamma = 2.666$. Už vieną žingsnį pasiekė minimumą tašką $X_{\min} = (0.3333, 0.3333)$. Apskaičiavo 2 gradientus ir 28 tikslo funkcijos $V(x, y)$ iškvietimų.

Žemiau pateikiu minimumo taško suradimo grafikus pradedant nuo X_m taško su skirtingomis intervalomis:



4.2) $X_m = (0, 0.4)$

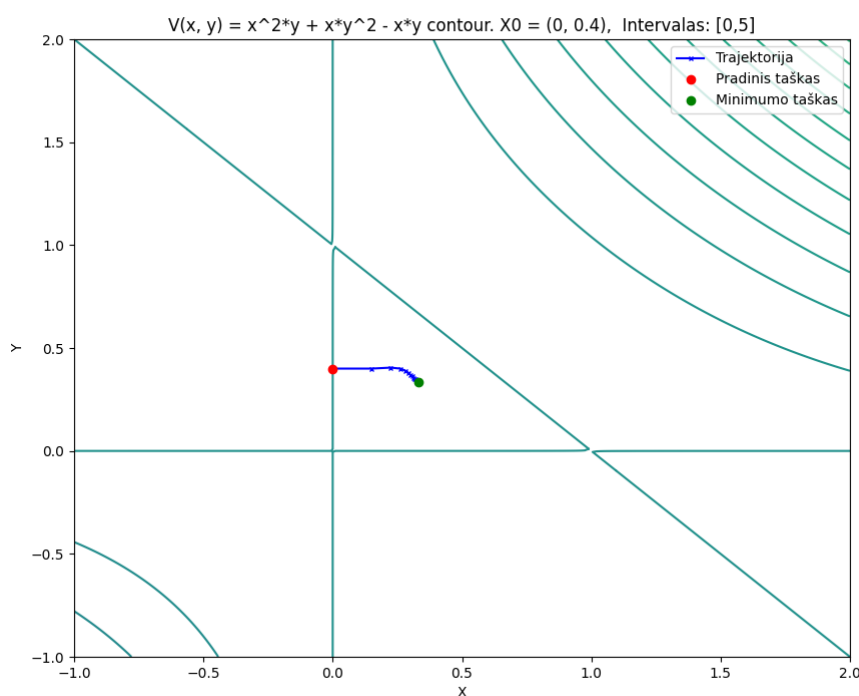
Greičiausio nusileidimo algoritmas, pradedant nuo $X_m = (0, 0.4)$, $\varepsilon = 0.0001$						
l	r	γ	$X_{\min}$	Iteracijų skaičius	V (x, y) iškvietimų skaičius	Gradientų skaičius
0	1	0.99996	(0.33169, 0.33499)	107	2462	108
0	5	4.99996	(0.33167, 0.33501)	19	495	20
0	10	9.99995	(0.33211, 0.33457)	8	217	9
5	10	9.99996	(0.33211, 0.33457)	8	209	9
10	20	...11.963	(0.33281, 0.33438)	7	194	8
5	30	...11.962	(0.33281, 0.33437)	7	205	8
20	30	...20.000	diverguoja į $-\infty$	-	-	-
1	3	2.99995	(0.33174, 0.33494)	34	817	35

... - reiskia, kad γ nepastovi ir keičiasi su kiekvieną iteraciją.

4.2.1) Aukso pjūvio algoritmo intervalas [0, 5]

Algoritmas baigė savo darbą per 19 iteracijų. Pasiekė minimumą tašką $x_{\min} = (0.33167, 0.33501)$. Visose 19 iteracijose $\gamma = 4.99996$. Algoritmas apskaičiavo 20 gradientus ir 495 tikslo funkcijos V (x, y) iškvietimų.

Grafikas, intervalas [0, 5]:



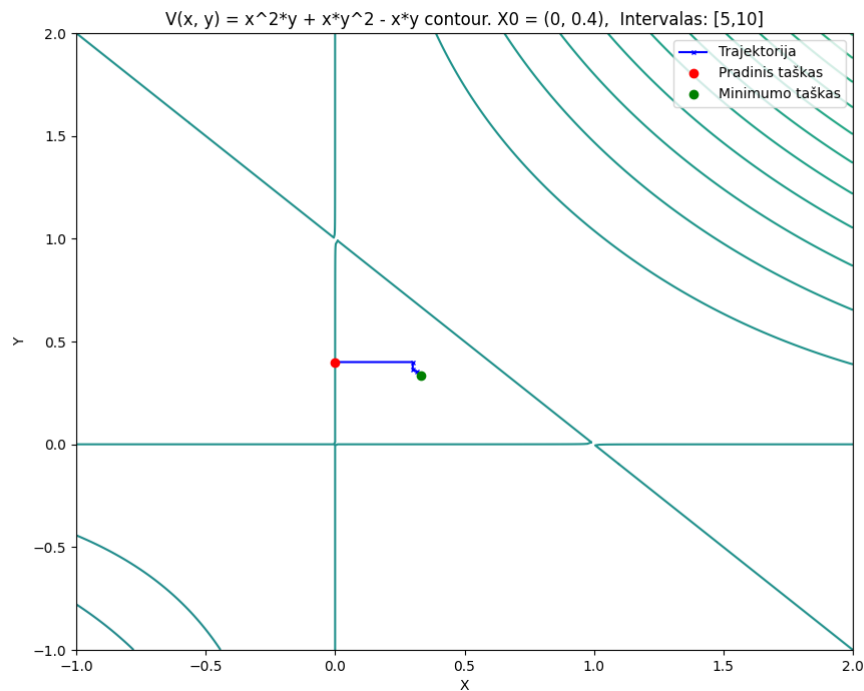
Šioje diagramoje pavaizduota gradientinio nusileidimo algoritmo trajektorija, kai naudojamas optimalus žingsnis, pradedant nuo pradinio taško $X_0 = (0, 0.4)$ ir judant link minimumo taško. Raudonas taškas žymi pradinį tašką, nuo kurio algoritmas pradeda savo kelionę, o mėlyna linija vaizduoja algoritmo trajektoriją – kelią, kuriuo algoritmas žingsnis po žingsnio artėja prie minimumo. Žalias taškas žymi minimumą, tai yra tašką, kuriame funkcijos vertė yra mažiausia, ir kur link algoritmas galiausiai susikoncentruoja. Šis grafikas iliustruoja, kaip gradientinis nusileidimas su optimaliu žingsniu efektyviai pasiekia minimumą, atsižvelgiant į nurodytą intervalą ir pradžios tašką.

4.2.2) Aukso pjūvio algoritmo intervalas [5, 10]

Algoritmas baigė savo darbą per 8 iteracijas. Pasiekė minimumą tašką $x_{\min} = (0.33211, 0.33457)$. Algoritmas apskaičiavo 9 gradientus ir 209 tikslo funkcijos $V(x, y)$ iškvietimų.

$X_m = (0, 0.4)$, intervalas [5, 10]				
Iteracija	$X_{\min}$	γ	Gradiento ilgis	Funkcijos $V(x, y)$ reikšmė:
0	(0.000000, 0.400000)	9.99996094960099	0.030000	0.000000
1	(0.299999, 0.400000)	9.99996094960099	0.003750	-0.004500
2	(0.300000, 0.362501)	9.99996094960099	0.001941	-0.004588
3	(0.316992, 0.353125)	9.99996094960099	0.001083	-0.004616
4	(0.322682, 0.343916)	9.99996094960099	0.000626	-0.004625
5	(0.327291, 0.339675)	9.99996094960099	0.000365	-0.004628
6	(0.329729, 0.336958)	9.99996094960099	0.000213	-0.004629
7	(0.331238, 0.335456)	9.99996094960099	0.000124	-0.004629
8	(0.332105, 0.334566)	9.99996094960099	0.000072	-0.004630

Grafikas, intervalas [5, 10]:



Ši trajektorija iliustruoja, kaip algoritmas palaipsniui artėja prie minimumo, mažindamas gradiento ilgį kiekviename žingsnyje, kad būtų pasiekta optimaliausia funkcijos vertė nurodytame intervale.

4.3) $X_0 = (0, 0)$

$$\nabla V(0, 0) = (0, 0) \Rightarrow |\nabla V(0, 0)| = 0 < \varepsilon$$

Algoritmas nepradės dirbti su tašku $(0, 0)$, nes gradiento ilgis šiame taške lygus 0, o tai yra mažiau už ε .

5) Deformuojamo simplekso algoritmas

Deformuojamo simplekso algoritmas konstruoja simpleksus ($n+1$ -mačio figūras), kad surasti minimumą tašką. Turime dviejų kintamųjų funkciją $V(x, y)$, taigi $n = 2$, ir algoritmas konstruos trikampus, kad rastų minimumo tašką.

Simplekso sudarymui reikia paduoti pradinį tašką X_0 ir koeficientą τ . Trikampio simplekso X_1 ir X_2 viršūnės galima surasti taikant šitas formules:

$$X_0 := (x_0, y_0)$$

$$X_1 = (x_0 + \delta_1, y_0 + \delta_2)$$

$$X_2 = (x_0 + \delta_2, y_0 + \delta_1)$$

kur δ_1 ir δ_2 :

$$\delta_1 = \frac{\sqrt{3} + 1}{2\sqrt{2}} \tau, \quad \delta_2 = \frac{\sqrt{3} - 1}{2\sqrt{2}} \tau$$

Po viršūnių sudarymų, algoritmas priskirs X_0, X_1, X_2 viršūnėms žymes:

- X_w (worst, blogiausia) — viršūnė su didžiausia $V(X)$ reikšme.
- X_b (best, geriausia) — viršūnė su mažiausia $V(X)$ reikšme.
- X_g (good, gera) — $V(X_b) < V(X_g) < V(X_w)$.

Toliau, algoritmas ieškos vidurio taško X_{mid} tarp X_g ir X_b :

$$X_{mid} = \frac{X_g + X_b}{2}$$

Simplekso deformavimui atliekamos 3 operacijos su tam tikrais koeficientais:

- Atspindys (angl. reflection), koeficientas $\alpha > 0$. Naujas taškas X_r :

$$X_r = (1 + \alpha)X_{mid} - \alpha X_w$$

- Išplėtimas (angl. expansion), koeficientas $\gamma > 0$. Naujas taškas X_e :

$$X_e = (1 - \gamma)X_{mid} + \gamma X_r$$

- Suspaudimas (angl. contraction), koeficientas $0 < \beta < 1$. Naujas taškas X_c :

$$X_c = \beta X_w + (1 - \beta)X_{mid}$$

Jei $V(X_c) \geq V(X_w)$ tada:

$$X_g = \frac{X_g + X_b}{2}$$

$$X_w = \frac{X_w + X_b}{2}$$

Algoritmas baigia savo darba kai funkcijos reiškių standartinis nuokrypis yra mažiau už $\epsilon = 0.0001$ arba kol algoritmas nepasieks iteracijos ribos. Maksimalus iteracijų skaičius = 250.

Parašiau Python programą (žr. Priedas 7.1), kuri randa lokalių minimumo tašką naudodama deformuojamo simplekso algoritimą (Python kode: def simplexMethod).

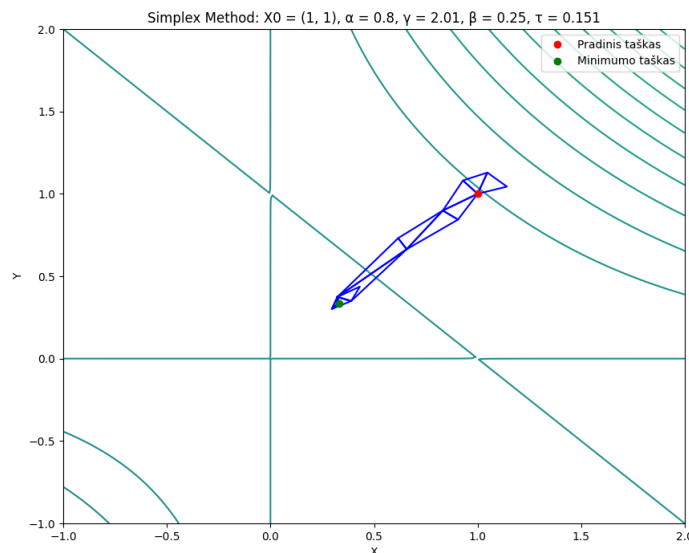
Po daugybės bandymų su Simpleksu algoritmu, radau optimalius koeficientus, kurie užtikrina efektyviausią ir stabiliausią algoritmo veikimą. Nustatyta, kad refleksijos koeficientas, optimaliai padeda judėti link minimumo, yra $\alpha = 0.8$, išplėtimo koeficientas, leidžiantis algoritmui efektyviai ieškoti geresnių sprendimų erdvėje, yra $\gamma = 2.01$, suspaudimo koeficientas, kuris padeda koreguoti prastų taškų padėtis, yra $\beta = 0.25$, o tau reikšmė yra $\tau = 0.151$. Šie koeficientai padeda algoritmui greičiau konverguoti link optimalaus sprendimo, tuo pačiu išlaikant stabilumą ir tikslumą.

5.1) $X_1 = (1, 1)$

$X_1 = (1, 1), \alpha = 0.8, \gamma = 2.01, \beta = 0.25, \tau = 0.151, \epsilon = 0.0001$				
Iteracija	Geriausią (minimalią) vertę	Funkcijos reikšmė geriausiam taškui:	Blogiausią (maksimalią) vertę	Funkcijos reikšmė blogiausiam taškui:
1	(1.000000, 1.000000)	0.125000	(1.141387, 1.043763)	0.176489
2	(1.000000, 1.000000)	0.125000	(1.047113, 1.128887)	0.173765
3	(0.832039, 0.898358)	0.068244	(0.929293, 1.080987)	0.126860
4	(0.832039, 0.898358)	0.068244	(1.000000, 1.000000)	0.125000
5	(0.657623, 0.663685)	0.017530	(0.905401, 0.843732)	0.071534
6	(0.657623, 0.663685)	0.017530	(0.832039, 0.898358)	0.068244
7	(0.323373, 0.373918)	-0.004575	(0.616375, 0.730853)	0.019552
8	(0.323373, 0.373918)	-0.004575	(0.657623, 0.663685)	0.017530
9	(0.323373, 0.373918)	-0.004575	(0.431844, 0.437076)	0.003093
...	...	...	...	...
18	(0.332802, 0.333397)	-0.004630	(0.333608, 0.330949)	-0.004629
19	(0.333389, 0.332894)	-0.004630	(0.333831, 0.333687)	-0.004630
20	(0.333280, 0.333281)	-0.004630	(0.332802, 0.333397)	-0.004630
21	(0.333280, 0.333281)	-0.004630	(0.333389, 0.332894)	-0.004630
22	(0.333280, 0.333281)	-0.004630	(0.333201, 0.333165)	-0.004630
23	(0.333280, 0.333281)	-0.004630	(0.333121, 0.333486)	-0.004630

Šioje lentelėje pavaizduota optimizacijos algoritmo eiga ieškant minimalaus taško naudojant Simplekso metodą. Pradinis taškas buvo nustatytas kaip $X_1 = (1, 1)$ su šiais koeficientais: refleksijos koeficientas $\alpha = 0.8$, išplėtimo koeficientas $\gamma = 2.01$, suspaudimo koeficientas $\beta = 0.25$, ir parametras $\tau = 0.151$. Sustabdymo kriterijus buvo nustatytas $\epsilon = 0.0001$. Best (Geriausioji vertė): Kiekvienoje iteracijoje „Best“ (geriausias) taškas nurodo tą tašką, kurio funkcijos reikšmė yra mažiausia arba minimali. Tai reiškia, kad šis taškas yra dabartinis optimizacijos proceso rezultatas, atitinkantis artimiausią minimumui vertę. Worst (Blogiausia vertė): blogiausias taškas rodo tašką su didžiausia arba maksimalia funkcijos verte, kurį algoritmas siekia pagerinti judėdamas link mažesnės vertės. Paskutinėje (23-ioje) iteracijoje algoritmas rado minimumo tašką, kurio koordinatės yra (0.333280, 0.333281). Tai reiškia, kad optimizacijos procesas baigėsi, kai skirtumas tarp „Best“ ir „Worst“ taškų tapo mažesnis už nustatytą ϵ , ir buvo pasiektas minimalus reikšmių skirtumas.

Grafikas:

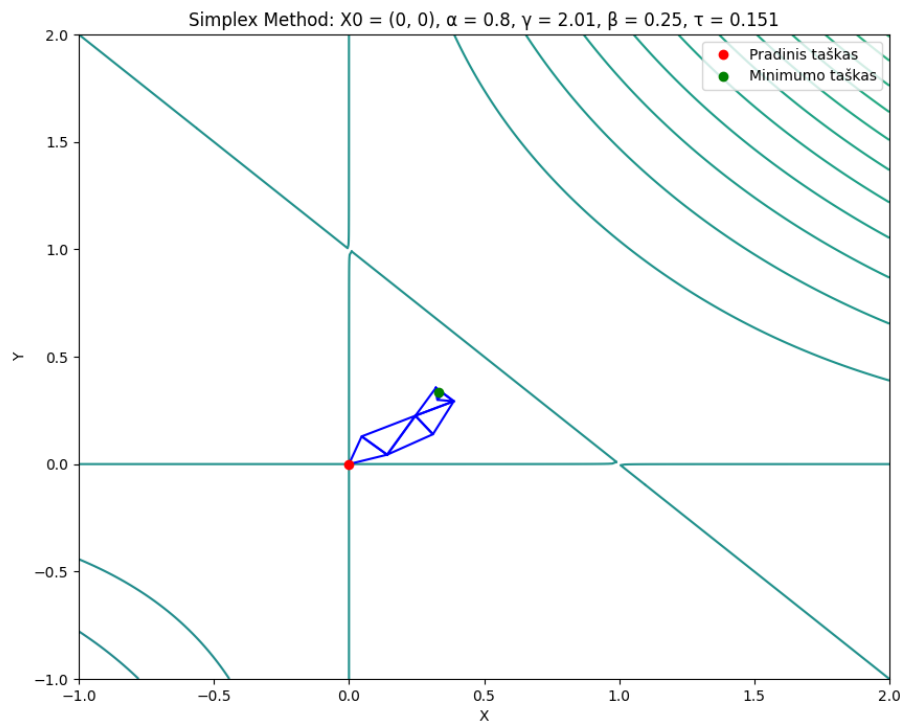


5.2) $X_0 = (0, 0)$

$X_0 = (0, 0)$, $\alpha = 0.8$, $\gamma = 2.01$, $\beta = 0.25$, $\tau = 0.151$, $\varepsilon = 0.0001$

Iteracija	Geriausią (minimalią) vertę	Funkcijos reikšmė geriausiam taškui:	Blogiausią (maksimalią) vertę	Funkcijos reikšmė blogiausiam taškui:
1	(0.141387, 0.043763)	-0.000630	(0.000000, 0.000000)	0.000000
2	(0.245804, 0.225136)	-0.003660	(0.047113, 0.128887)	-0.000625
3	(0.245804, 0.225136)	-0.003660	(0.141387, 0.043763)	-0.000630
4	(0.387817, 0.292621)	-0.004533	(0.310781, 0.138900)	-0.002969
5	(0.321634, 0.354861)	-0.004615	(0.245804, 0.225136)	-0.003660
6	(0.321634, 0.354861)	-0.004615	(0.387817, 0.292621)	-0.004533
7	(0.340378, 0.318387)	-0.004623	(0.327495, 0.299090)	-0.004572
8	(0.330128, 0.327240)	-0.004627	(0.321634, 0.354861)	-0.004615
9	(0.331848, 0.330826)	-0.004629	(0.340378, 0.318387)	-0.004623
...	...	...	...	...
21	(0.333413, 0.333333)	-0.004630	(0.333461, 0.333124)	-0.004630
22	(0.333413, 0.333333)	-0.004630	(0.333542, 0.333216)	-0.004630
23	(0.333364, 0.333340)	-0.004630	(0.333473, 0.333237)	-0.004630
24	(0.333364, 0.333340)	-0.004630	(0.333413, 0.333333)	-0.004630
25	(0.333364, 0.333340)	-0.004630	(0.333320, 0.333416)	-0.004630
26	(0.333328, 0.333346)	-0.004630	(0.333285, 0.333414)	-0.004630

Pradinis taškas buvo nustatytas kaip $X_0 = (0, 0)$. Paskutinėje (26-ioje) iteracijoje algoritmas rado minimumo tašką, kurio koordinatės yra $(0.333328, 0.333346)$. Tai reiškia, kad optimizacijos procesas baigėsi, kai skirtumas tarp „Best“ ir „Worst“ taškų tapo mažesnis už nustatytą ε , ir buvo pasiektas minimalus reikšmių skirtumas.

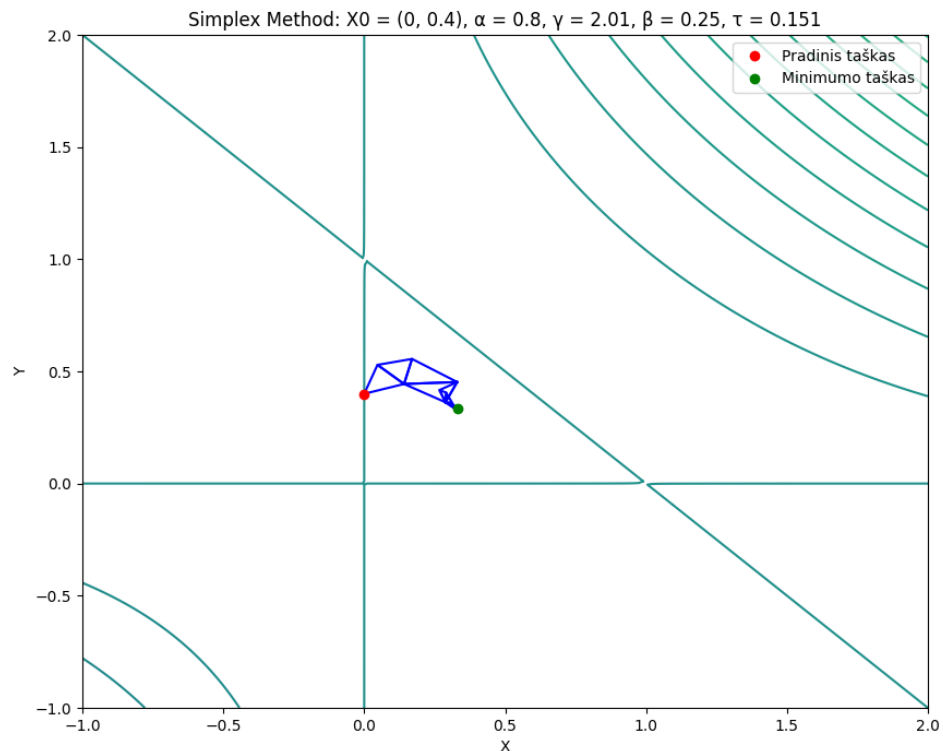


5.3) $X_m = (0, 0.4)$

$X_m = (0, 0.4)$, $\alpha = 0.8$, $\gamma = 2.01$, $\beta = 0.25$, $\tau = 0.151$, $\varepsilon = 0.0001$

Iteracija	Geriausią (minimalią) vertę	Funkcijos reikšmė geriausiam taškė:	Blogiausią (maksimalią) vertę	Funkcijos reikšmė blogiausiam taškė:
1	(0.141387, 0.443763)	-0.003254	(0.000000, 0.400000)	0.000000
2	(0.141387, 0.443763)	-0.003254	(0.047113, 0.528887)	-0.001321
3	(0.329833, 0.452440)	-0.004061	(0.169650, 0.555385)	-0.003238
4	(0.288378, 0.362275)	-0.004562	(0.141387, 0.443763)	-0.003254
5	(0.288378, 0.362275)	-0.004562	(0.329833, 0.452440)	-0.004061
6	(0.288378, 0.362275)	-0.004562	(0.267176, 0.416459)	-0.004400
7	(0.325617, 0.331037)	-0.004626	(0.290791, 0.405135)	-0.004478
8	(0.325617, 0.331037)	-0.004626	(0.288378, 0.362275)	-0.004562
9	(0.335005, 0.333262)	-0.004630	(0.302946, 0.361276)	-0.004594
...	...	...	...	...
16	(0.333539, 0.333583)	-0.004630	(0.334138, 0.332881)	-0.004630
17	(0.333452, 0.333268)	-0.004630	(0.332906, 0.333209)	-0.004630
18	(0.333348, 0.333371)	-0.004630	(0.333539, 0.333583)	-0.004630
19	(0.333348, 0.333371)	-0.004630	(0.333435, 0.333385)	-0.004630
20	(0.333348, 0.333371)	-0.004630	(0.333452, 0.333268)	-0.004630
21	(0.333287, 0.333360)	-0.004630	(0.333372, 0.333267)	-0.004630

Pradinis taškas buvo nustatytas kaip $X_m = (0, 0.4)$. Paskutinėje (21-ioje) iteracijoje algoritmas rado minimumo tašką, kurio koordinatės yra $(0.333287, 0.333360)$. Tai reiškia, kad optimizacijos procesas baigėsi, kai skirtumas tarp „Best“ ir „Worst“ taškų tapo mažesnis už nustatytą ε , ir buvo pasiektas minimalus reikšmių skirtumas.



6) Išvada

Išanalizavus visus taikytus optimizavimo algoritmus, galima padaryti keletą svarbių išvadų apie jų efektyvumą ir taikymo sritis. Gradientinio nusileidimo algoritmas ir greičiausiojo nusileidimo algoritmas pasirodė esą veiksmingi ieškant funkcijos minimumo, tačiau kiekvienas iš jų turi savo privalumų ir trūkumų.

Gradientinis nusileidimas su fiksuotu žingsnio dydžiu yra paprastas ir greitas, tačiau jo sėkmė labai priklauso nuo tinkamo žingsnio dydžio γ pasirinkimo. Netinkamai parinktas γ gali lemti lėtą konvergenciją arba netgi algoritmo nesugebėjimą rasti minimumo. Greičiausiojo nusileidimo algoritmas, kita vertus, kiekvienoje iteracijoje adaptuoja žingsnio dydį, naudodamas optimalaus žingsnio paiešką (pvz., auksinio pjūvio metodą). Tai leidžia jam greičiau ir tiksliau konverguoti į minimumą, nepriklausomai nuo pradinio žingsnio dydžio pasirinkimo. Tačiau šis algoritmas dažnai reikalauja daugiau skaičiavimų per iteraciją dėl papildomų optimizavimo žingsnių.

Kalbant apie simplekso metodą, jis pasirodė esąs naudingas tais atvejais, kai gradientiniai metodai neefektyvūs arba netaikomi, pavyzdžiui, kai funkcija nėra diferencijuojama arba gradientas lygus nuliui pradiniam taške. Simplekso metodas nenaudoja gradiento informacijos, todėl gali rasti minimumą net ir sudėtingesnėse situacijose. Tačiau jis dažnai reikalauja daugiau iteracijų ir daugiau funkcijos įvertinimų, kad pasiektų panašų tikslumą kaip gradientiniai metodai. Be to, jo konvergencija gali būti lėtesnė, ypač esant didesniai kintamųjų skaičiui.

Apibendrinant, gradientiniai metodai yra efektyvūs ir greiti, kai funkcija yra lygi ir diferencijuojama, o tinkamas žingsnio dydis gali būti parinktas arba adaptuotas. Greitojo nusileidimo algoritmas suteikia pranašumą dėl adaptacinio žingsnio dydžio, tačiau už tai moka didesniais skaičiavimo ištekliais per iteraciją. Simplekso metodas yra lankstesnis ir gali būti taikomas platesniam problemų spektrui, tačiau dažnai yra mažiau efektyvus skaičiavimo atžvilgiu. Renkantis optimizavimo metodą, svarbu atsižvelgti į funkcijos savybes, pradinio taško pasirinkimą ir norimą tikslumą. Gradientiniai metodai yra rekomenduojami, kai funkcija yra gerai suprantama ir diferencijuojama, o simplekso metodas gali būti geresnis pasirinkimas, kai funkcija yra sudėtinga arba gradientinė informacija nėra lengvai prieinama.

7) Priedas, kodas

7.1) optimization_algorithms.py, tai kodas su visų algoritmų realizacijomis.

```
import math
import numpy as np
import statistics
import pandas as pd

# Vector class as per your implementation
class Vector(object):
    def __init__(self, x, y): # konstruktorius
        self.x = x # x saugomas kaip vektoriaus koordinate
        self.y = y

    def length(self):
        return math.sqrt(self.x ** 2 + self.y ** 2) # pitagoro teorema

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __radd__(self, other):
        return self.__add__(other) # atvirkstine

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __rsub__(self, other):
        return self.__sub__(other)

    def __mul__(self, other: float):
        return Vector(self.x * other, self.y * other)

    def __rmul__(self, other: float):
        return self.__mul__(other)

    def __truediv__(self, other: float):
        ratio = 1.0 / other
        return self * ratio

    def __rtruediv__(self, other: float):
        return self.__truediv__(other)

    def __str__(self):
        return "{0}, {1}".format(self.x, self.y)

# Global variables for counting
gradientCount = 0
functionCalls = 0

gamma = 0.03 #kaip toli judesim

# Gradient Descent
```

```

def gradient_descent(f, grad_f, X0, step_size=gamma, epsilon=0.0001,
max_iter=10000, return_table=False):
    global gradientCount
    gradientCount = 0
    X = Vector(*X0) # pradinė vektoriaus reikšmė
    grad = grad_f(X)
    iterations = 0
    trajectory = [X]
    table_data = []
    # step_size = gamma # Dabar step_size yra perduodamas iš išorės

    while grad.length() >= epsilon and iterations < max_iter:
        f_X = f(X)
        table_data.append({
            'Iteration': iterations,
            'X1': X.x,
            'X2': X.y,
            'f(X)': f_X,
            'Gradient Norm': grad.length()
        })
        X = X - step_size * grad # gradiento nusileidimo žingsnis,
        atnaujinaime taska juodami prieš gradiento krypti
        grad = grad_f(X)
        iterations += 1
        trajectory.append(X)

    f_X = f(X) # f-jos reikšmė paskutiniame taške
    table_data.append({
        'Iteration': iterations,
        'X1': X.x,
        'X2': X.y,
        'f(X)': f_X,
        'Gradient Norm': grad.length()
    })

    if return_table:
        table = pd.DataFrame(table_data)
        return X, f_X, iterations, trajectory, table
    else:
        return X, f_X, iterations, trajectory

# Gradient Descent with Optimal Step Size
def gradient_descent_optimal(f, grad_f, X0, epsilon=0.000001,
max_iter=10000, return_table=False):
    global gradientCount, functionCalls
    gradientCount = 0
    functionCalls = 0
    X = Vector(*X0)
    grad = grad_f(X)
    iterations = 0
    trajectory = [X]
    table_data = []

    tau = 0.618

    def golden_ratio_search(f_line, l, r, epsilon):
        L = r - l

```



```

x1 = r - tau * L
x2 = l + tau * L

fx1 = f_line(x1)
fx2 = f_line(x2)

while L >= epsilon:
    if fx2 < fx1:
        l = x1
        L = r - l
        x1 = x2
        x2 = l + tau * L

        fx1 = fx2
        fx2 = f_line(x2)
    else:
        r = x2
        L = r - l
        x2 = x1
        x1 = r - tau * L

        fx2 = fx1
        fx1 = f_line(x1)

return (l + r) / 2.0 #optimalus zingsnis

while grad.length() >= epsilon and iterations < max_iter:
    f_X = f(X)
    table_data.append({
        'Iteration': iterations,
        'X1': X.x,
        'X2': X.y,
        'f(X)': f_X,
        'Gradient Norm': grad.length()
    })

    def f_line(step):#tam kad rasti optimalu zingsni, ir tai
minimizuojama funkcija
        new_X = X - step * grad #new_X yra pasislinkes nuo
dabartinio tasko X gradiento kryptimi per atstuma step
        return f(new_X)

    step_size = golden_ratio_search(f_line, 0, 1, epsilon)
    #print(step_size)
    X = X - step_size * grad #judame prieš gradientą
    grad = grad_f(X) #naujas gradiendas taske x
    iterations += 1
    trajectory.append(X)

f_X = f(X)
table_data.append({
    'Iteration': iterations,
    'X1': X.x,
    'X2': X.y,
    'f(X)': f_X,
    'Gradient Norm': grad.length()
})

```

```

    if return_table:
        table = pd.DataFrame(table_data)
        return X, f_X, iterations, trajectory, table
    else:
        return X, f_X, iterations, trajectory

# Simplex method
def simplexMethod(f, X0, reflectionRate: float = 1, expansionRate: float
= 2.0, contractionRate: float = 0.5,
                    tau: float = 1.0, epsilon: float = 1e-8, maxIterations:
int = 250, return_table=False):
    constructedSimplexes = []
    table_data = []
    step = 0.025
    #reflectionRate - alfa, refleksijos koeficientas
    #expansionRate - gamma, ispletimo koeficientas
    #contractionRate - beta, kontrakcijos koeficientas, suspausdimas
    def contract():
        nonlocal worst, fWorst, good, fGood, mid

        contractedPoint = (1 - contractionRate) * mid + contractionRate *
worst #perkelti blogiausia taska link centroido, taip sumazinant
simplekso dydi
        fContr = f(contractedPoint)

        if fContr < fWorst:
            worst, fWorst = contractedPoint, fContr
        else: #shrink aplink best taska
            good = (good + best) / 2
            fGood = f(good)

            worst = (worst + best) / 2
            fWorst = f(worst)

    # Koeficientai, kad simpleksas būtų lygiašonis trikampis
    b1 = ((math.sqrt(2) + 1) / (2 * math.sqrt(2))) * tau
    b2 = ((math.sqrt(2) - 1) / (2 * math.sqrt(2))) * tau

    X1 = Vector(X0.x + step + b2, X0.y + b1) # nauji vektoriai
    X2 = Vector(X0.x + step / 2 + b1, X0.y + step * 0.866 + b2)

    simplex = {X0: f(X0), X1: f(X1), X2: f(X2)} #zodynas raktas:reiksme
    iterations = 0

    # max atstumas
    def simplex_size(simplex_points):
        max_distance = 0
        for i in range(len(simplex_points)):
            for j in range(i + 1, len(simplex_points)):
                distance = (simplex_points[i] -
simplex_points[j]).length()
                if distance > max_distance:
                    max_distance = distance
        return max_distance

    while iterations < maxIterations:

```

```

iterations += 1

points = sorted(simplex.items(), key=lambda x: x[1])

best, fBest = points[0][0], points[0][1]
good, fGood = points[1][0], points[1][1]
worst, fWorst = points[2][0], points[2][1]

constructedSimplexes.append([best, good, worst])

# Collect iteration data
table_data.append({
    'Iteration': iterations,
    'Best X1': best.x,
    'Best X2': best.y,
    'f(Best)': fBest,
    'Worst X1': worst.x,
    'Worst X2': worst.y,
    'f(Worst)': fWorst
})

mid = (best + good) / 2.0 #centroido skaiciavimas
refl = mid + reflectionRate * (mid - worst) #refleksija
fRefl = f(refl)

if fRefl < fBest:
    exp = mid + expansionRate * (refl - mid)
    fExp = f(exp)
    if fExp < fRefl:
        worst, fWorst = exp, fExp
    else:
        worst, fWorst = refl, fRefl
elif fRefl < fGood:
    worst, fWorst = refl, fRefl
else:
    contract() #refleksija nesuteikia geresnio tasko fRefl >=
fGood, todel darom suspausdima

if simplex_size([best, good, worst]) < epsilon:
    break

simplex = {worst: fWorst, good: fGood, best: fBest}

if return_table:
    table = pd.DataFrame(table_data)
    return best, iterations, table, constructedSimplexes
else:
    return best, iterations, constructedSimplexes

```

7.2) optimization_algorithms.py, tai kodas vizualizacijai, rezultatams

```
import math
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from optimization_algorithms import (
    gradient_descent,
    gradient_descent_optimal,
    simplexMethod,
    Vector,
    gamma
)

# Global variables for counting
gradientCount = 0
functionCalls = 0

# Objective Function
def f(X):
    global functionCalls
    functionCalls += 1
    x, y = X.x, X.y
    return 0.125 * (x ** 2 * y + x * y ** 2 - x * y)

# Gradient of the Objective Function
def grad_f(X):
    global gradientCount
    gradientCount += 1
    x, y = X.x, X.y
    df_dx = 0.125 * (2 * x * y + y ** 2 - y)
    df_dy = 0.125 * (x ** 2 + 2 * x * y - x)
    return Vector(df_dx, df_dy)

# Starting Point
X0 = (1, 1)
X0_vector = Vector(*X0)

print("\nFunction and gradient at X0:")
print("X0 =", X0_vector)
print("f(X0) =", f(X0_vector))
print("grad_f(X0) =", grad_f(X0_vector))

epsilon = 1e-4

results = {}

# Gradient Descent
gradientCount = 0
functionCalls = 0

X_gd, f_gd, steps_gd, traj_gd, table_gd = gradient_descent(
    f, grad_f, X0, gamma, epsilon, max_iter=10000, return_table=True
)
func_calls_gd = functionCalls
grad_calls_gd = gradientCount
```

```

print(
    f"\nGradient Descent (Fixed Step Size): X = {X_gd}, f(X) = {f_gd},
    steps = {steps_gd}, "
    f"function calls = {func_calls_gd}, gradient evaluations =
    {grad_calls_gd}"
)

# Store results
results["Gradient Descent"] = {
    "X": X_gd,
    "f(X)": f_gd,
    "steps": steps_gd,
    "function_calls": func_calls_gd,
    "gradient_calls": grad_calls_gd,
    "trajectory": traj_gd,
    "table": table_gd,
}

# Gradient Descent with Optimal Step Size
gradientCount = 0
functionCalls = 0
X_sd, f_sd, steps_sd, traj_sd, table_sd = gradient_descent_optimal(
    f, grad_f, X0, epsilon, max_iter=10000, return_table=True
)
func_calls_sd = functionCalls
grad_calls_sd = gradientCount
print(
    f"\nGradient Descent with Optimal Step Size: X = {X_sd}, f(X) =
    {f_sd}, steps = {steps_sd}, "
    f"function calls = {func_calls_sd}, gradient evaluations =
    {grad_calls_sd}"
)

# Store results
results["Gradient Descent with Optimal Step Size"] = {
    "X": X_sd,
    "f(X)": f_sd,
    "steps": steps_sd,
    "function_calls": func_calls_sd,
    "gradient_calls": grad_calls_sd,
    "trajectory": traj_sd,
    "table": table_sd,
}

# Simplex Method
functionCalls = 0
X0_vector = Vector(*X0)
reflection = 0.8
expansion = 2.01
contraction = 0.25
tau = 0.151

best_point, iterations, table_ds, constructedSimplexes = simplexMethod(
    f,
    X0_vector,
    reflectionRate=reflection,
    expansionRate=expansion,

```

```

        contractionRate=contraction,
        tau=tau,
        epsilon=epsilon,
        maxIterations=250,
        return_table=True,
    )

func_calls_ds = functionCalls
print(
    f"\nSimplex Method: X = {best_point}, f(X) = {f(best_point)}, steps = {iterations}, function calls = {func_calls_ds}"
)

# Store results
results["Simplex Method"] = {
    "X": best_point,
    "f(X)": f(best_point),
    "steps": iterations,
    "function_calls": func_calls_ds,
    "gradient_calls": "N/A",
    "trajectory": constructedSimplexes,
    "table": table_ds,
}

# Comparison Table
comparison_data = []
for algo_name, res in results.items():
    comparison_data.append(
        {
            "Algorithm": algo_name,
            "Obtained Solution": f"({round(res['X'].x, 5)}, {round(res['X'].y, 5)})",
            "Estimated Minimum": round(res["f(X)"], 5),
            "Number of Steps": res["steps"],
            "Function Evaluations": res["function_calls"],
            "Gradient Evaluations": res.get("gradient_calls", "N/A"),
        }
    )
comparison_table = pd.DataFrame(comparison_data)

print("\nComparison of Results:")
print(comparison_table.to_string(index=False))

# Print Iteration Tables
print(f"\nIteration details starting from X0:")
for algo_name, res in results.items():
    if "table" in res:
        table = res["table"]
        print(f"\nAlgorithm: {algo_name}")
        print(table.to_string(index=False)) # Print the table without the index

# Visualization Functions
def plot_algorithm_with_contour(algo_name, traj, x0, step_size=None):
    # Pagal pateiktą pavyzdį
    x = np.linspace(-1, 2, 200)
    y = np.linspace(-1, 2, 200)

```

```

X, Y = np.meshgrid(x, y)
Z = 0.125 * (X ** 2 * Y + X * Y ** 2 - X * Y)
levels = np.linspace(-5, 5, 75)

plt.figure(figsize=(10, 8))
plt.contour(X, Y, Z, levels=levels)

allX = [point.x for point in traj]
allY = [point.y for point in traj]

plt.plot(allX, allY, '-bx', label='Trajektorija', markersize=3)

# Pradinis ir minimumo taškai
plt.plot(x0.x, x0.y, 'ro', label='Pradinis taškas')
min_point = traj[-1]
plt.plot(min_point.x, min_point.y, 'go', label='Minimumo taškas')

plt.xlabel('X')
plt.ylabel('Y')
plt.title("V(x, y) = x^2*y + x*y^2 - x*y contour. X0 = {0}, y = {1}".format(x0, step_size))
# plt.title("V(x, y) = x^2*y + x*y^2 - x*y contour. X0 = {0}, Intervalas: [5,10]".format(x0))
plt.legend()
plt.show()

def plot_simplex_with_contour(constructedSimplexes, x0, reflection, expansion, contraction, tau):
    # Pagal pateiktą pavyzdį
    x = np.linspace(-1, 2, 200)
    y = np.linspace(-1, 2, 200)
    X, Y = np.meshgrid(x, y)
    Z = 0.125 * (X ** 2 * Y + X * Y ** 2 - X * Y)
    levels = np.linspace(-5, 5, 75)

    plt.figure(figsize=(10, 8))
    plt.contour(X, Y, Z, levels=levels)

    for simplex in constructedSimplexes:
        simplexX = [vertex.x for vertex in simplex] + [simplex[0].x]
        simplexY = [vertex.y for vertex in simplex] + [simplex[0].y]
        plt.plot(simplexX, simplexY, '-b')

    plt.plot(x0.x, x0.y, 'ro', label='Pradinis taškas')
    final_simplex = constructedSimplexes[-1]
    best_vertex = final_simplex[0]
    plt.plot(best_vertex.x, best_vertex.y, 'go', label='Minimumo taškas')

    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title("Simplex Method: X0 = {0}, α = {1}, γ = {2}, β = {3}, τ = {4}".format(
        x0, reflection, expansion, contraction, tau
    ))
    plt.legend()
    plt.show()

```

```
# Vizualizacija
plot_algorithm_with_contour(
    "Gradient Descent",
    results["Gradient Descent"]["trajectory"],
    X0_vector,
    gamma
)

plot_algorithm_with_contour(
    "Gradient Descent with Optimal Step Size",
    results["Gradient Descent with Optimal Step Size"]["trajectory"],
    X0_vector
)

plot_simplex_with_contour(
    results["Simplex Method"]["trajectory"],
    X0_vector,
    reflection,
    expansion,
    contraction,
    tau
)
```